THE GENERATIVIZATION SERIES · BOOK I

Agentic Loops

Building Software Where Models Decide What Happens Next

Chapter 1: One Small Step

This is a free sample chapter from Agentic Loops, by José Luis Martin Matias, typeset exactly as it appears in the book. The lab the chapter ends with, labs/ch01_one_small_step_lab.py, is in the same repository and runs without an API key or any third-party package.

https://github.com/jmartinmatias/agentic-loops

© 2026 José Luis Martin Matias. All rights reserved. The code listings are released under the MIT License.

CHAPTER 1

One Small Step

Why the loop is the program

> *“Program alarm.”*
>
> *“It’s a 1202.”*
>
> Neil Armstrong, descending, July 20, 1969

Five minutes into the final descent, with the Moon filling the windows and the engine burning, the computer flying Apollo 11’s lunar module said, in effect: *I have too much to do.*

It said it as a number. **1202** flashed onto the display, a program alarm, a code no astronaut had a page for, and the master alarm sounded in the cabin. Neil Armstrong asked Houston for a reading on it. He and Buzz Aldrin were dropping toward the Sea of Tranquility in a machine with less memory than a short email, and the machine had just interrupted its own landing to complain.

You will not appreciate what happened next until you know what was happening inside the box.

The Apollo Guidance Computer was, by any modern measure, almost nothing: around seventy pounds of hand-wired electronics, a couple of kilobytes of erasable memory, a clock several thousand times slower than the phone in your pocket. Its software had been written over years at the MIT Instrumentation Laboratory by a team whose software engineering division was led by Margaret Hamilton. Written, and then literally *woven*, thread by thread through magnetic cores, by textile workers whose finished fabric was the program. If you wanted to change a line of code, you re-wove the rope.

And at its heart, that software was a loop. Not a metaphorical loop: an executive scheduler, built on a design by Hal Laning, that cycled through a queue of jobs in strict priority order. Read the radar. Update the trajectory. Fire the thrusters. Refresh the display. Around and around, many times a second, each job handwritten, each

transition known in advance, the whole choreography fixed months before launch and frozen (in the ropes, literally) for the flight.

The 1202 alarm meant the loop was drowning. A checklist decision had left the rendezvous radar, the one you would need to find the command module again in an emergency, switched on during descent, and a subtle electrical mismatch made it flood the computer with meaningless interrupts, stealing roughly fifteen percent of its processing time. Fifteen percent does not sound fatal. But the descent schedule had been budgeted close to the bone, and the executive was now being asked to do more work per cycle than the cycle contained. Its job queue overflowed. Hence the alarm: 1202, executive overflow.

Here is the part that matters. The computer did not crash. It did not freeze, did not corrupt the trajectory, did not do the thing every machine you have ever owned does when it runs out of headroom. It executed a plan its authors had written for exactly this moment, years earlier, on the ground: it **restarted itself, flushed everything from the queue, and rebuilt only the jobs that mattered**, highest priority first. Steering the descent: kept. Navigation: kept. The display refresh and the low-priority housekeeping: dropped without ceremony. The computer triaged itself, mid-landing, in a fraction of a second, and kept flying.

In Mission Control, the question "GO or abort?" landed on a guidance officer named Steve Bales, age twenty-six, who turned to a backroom engineer named Jack Garman, age twenty-four. Weeks earlier, in a simulation, this exact family of alarms had been thrown at the team and they had called an abort. Wrongly, the debrief concluded. So Garman had done a very human thing: he had written out, by hand, a cheat sheet of every program alarm and what it actually meant. He looked at his list. As long as the alarms did not come continuously, the computer was doing precisely what it was designed to do. Bales called GO. Capcom Charlie Duke passed it up ("we're GO on that alarm"), and the alarms kept coming, four more of them, all the way down through the pitchover, while Armstrong took over the final approach by hand to skip past a boulder field, and Eagle landed with well under a minute of hover fuel to spare.

Half a billion people heard "one small step." Almost none of them heard the other sentence, the one this book is built on, the reason a twenty-six-year-old could bet two lives and a decade of national effort on a machine that was actively complaining:

He knew what the loop would do.

Every job the computer could run had been written by a person. Every transition between jobs had been decided in advance. And when the authored behavior hit

conditions nobody had scripted, a *separate layer* (priorities, restart tables, a watchdog) guaranteed the outcome anyway. The behavior was frozen. The guarantees were locked. That is what made GO a rational word.

This book is about what happens when we thaw the first of those layers: when the next step of a program is *generated* by a model at runtime instead of written by a person in advance. It is a genuinely new way to build software, and it is the reason you picked this book up. But you cannot understand what changes until you can name, precisely, what used to be fixed. So this first chapter is about the machine we are leaving: the classical loop, in full. We will take it apart, name its four layers, meet it in the wild, and build one: a small, honest, deterministic loop that we will spend the rest of the book melting, one layer at a time.

By the end of the chapter you will own four words (**body**, **actor**, **flow**, **guarantee**) and they will carry you through five books.

1.1 The Oldest Trick in Computing

Strip away sixty years of frameworks and there are only a handful of shapes a program can take. In 1966 (the same year, pleasingly, that a chatbot first fooled a secretary, but that is Chapter 2) two Italian computer scientists, Corrado Böhm and Giuseppe Jacopini, proved a result now folded so deeply into practice that nobody cites it anymore: *any* computation can be expressed with just three structures. Do this, then that (**sequence**). Do this *or* that (**selection**). Do this *again* (**iteration**).

Sequence and selection are clerks. They execute and they choose, and then they are finished. Iteration is different in kind. Iteration is the only structure that says *keep going until something is true*, the only one that lets a fixed, finite text produce unbounded, open-ended work. A thousand lines of woven rope flew a spacecraft for days, not because the rope was long, but because it looped.

That is the trick, and it is genuinely the oldest one. The loop is where a program stops being a recipe and starts being a *process*: a thing that persists, watches, responds, and, critically, must one day decide it is done. Recursion, the loop's elegant cousin, plays the same trick wearing academic dress: a function that continues by invoking itself, with a base case standing where the loop's exit condition would stand. Same trick, same obligations.

Two obligations, to be exact, and they will follow us through every chapter of every book in this series:

1. **Something must change each time around.** A loop that revisits the same state has stopped computing and started spinning. (Hold that thought until section 1.7, where we will watch it happen.)
2. **Something must eventually be true.** A loop with no reachable exit condition is not a process; it is a hostage situation. The sorcerer's apprentice enchants the broom in Chapter 10, but the broom is already standing in the corner of this one.

Everything else (the schedulers, the event systems, the orchestration frameworks, the agent runtimes that this series is really about) is engineering wrapped around those two obligations.

1.2 Programs as State Transitions

Here is the least glamorous and most load-bearing idea in this book. A running program, at any instant, can be described by its **state**: everything it knows and everything it has done that still matters. And a loop is nothing more than a rule applied to that state, over and over:

```
while not finished(state):
    state = f(state)
```

That is it. That is the physics. `f`, the **transition function**, takes the world as the program understands it and returns the world one step later. Everything a loop will ever do is contained in three decisions someone made in advance: what counts as state, what `f` does, and what `finished` means.

Look at that tiny program long enough and it separates into layers, the way a landscape separates into geology. Four questions, four layers:

What happens during a step? That is the **body**: the contents of `f`. On Eagle, the body was guidance equations and thruster commands, derived by engineers and checked by other engineers. In your codebase, it is the functions the loop calls.

Who, or what, performs the step? That is the **actor**: the thing with an identity, capabilities, and permissions that executes the body. The AGC and its scheduled jobs. Your worker process. The service account it runs as.

How are steps arranged over time? That is the **flow**: the order, the branching, the scheduling, the choreography. Laning's priority executive. Your pipeline definition. The arrow in every architecture diagram you have ever drawn.

What must remain true, no matter what? That is the **guarantee**: the layer that does not *do* anything. It *forbids* and *verifies*. The priority ordering that said steering outranks displays. The restart tables. Garman's handwritten sheet, which was a guarantee wearing a human face.

Draw them as a stack and label the Apollo column, because we will be redrawing this picture for five books:

```
 layer      | on Eagle, July 1969             | authored when?
------------+---------------------------------+---------------
 GUARANTEE  | priorities, restart, watchdogs, |  design time
            | Garman's list, Bales's GO rules |   (LOCKED)
 FLOW       | the executive's schedule        |  design time
 ACTOR      | the AGC and its jobs            |  design time
 BODY       | guidance equations, commands    |  design time
```

Read the right-hand column and notice what is remarkable about classical software, precisely because nobody ever remarks on it: **every layer is authored before execution.**

The body, the actor, the flow: all of them are *artifacts*, fixed in advance, as thoroughly frozen as software woven into rope. At runtime, nothing new comes into existence; the machine merely *replays* decisions people already made. This is why Bales could say GO. It is why your on-call engineer can read a stack trace. Determinism is not a feature of classical software; it is the medium the whole thing is sculpted in.

Keep the word **frozen** close. In section 1.4 we will make it precise, and in Chapter 2 we will apply heat.

Four words for five books

Body: what happens during a computational step.
Actor: who or what performs the step.
Flow: how steps and actors are arranged over time.
Guarantee: what must remain true, or be verified, regardless of the other three.

This series tells one story: the first three layers, one by one, stop being written in advance and start being generated at runtime, while the fourth absorbs the responsibility they leave behind. This book melts the body. Its sequels melt the actor, then the flow. The guarantee is the layer we will fight, for five books, to keep frozen.

1.3 Everything Is Secretly a Loop

Once you have the shape, you see it everywhere. And I do mean everywhere, because the software industry has spent sixty years rediscovering the loop and each time giving it a more dignified name.

Cron is a loop with a wristwatch: wake, check the schedule, run what is due, sleep.

An ETL pipeline is a loop wearing a lanyard: extract, transform, load, and back tomorrow at 02:00 to do it again. **A web server** is a loop that answers the door: accept a connection, handle the request, return to the doorway. The fact that it may be doing so on ten thousand doors at once changes the engineering enormously and the shape not at all. **A game engine** runs the most honest loop in the business: read input, update the world, draw the world, sixty times a second, forever, with a frame budget so strict it makes Apollo's executive look leisurely. **The event loop** in your browser and in Node.js is a loop that has outsourced its to-do list: it does not know what work is coming, only how to take the next callback from the queue and run it. **A thermostat**, and every control system descended from it up to and including the one that flew Eagle, is a loop with a grievance: measure the world, compare it to the world you wanted, act to shrink the difference, measure again.

And **workflow engines** (the Airflows, Step Functions, and Temporal clusters of the world, the tools closest to where this series is heading) are loops that have been promoted into management. You hand them a graph of tasks, and underneath the DAGs and the retry policies there is a scheduler doing exactly what Laning's executive did: cycle through pending work, pick the next eligible job, dispatch it, record what happened, go around again.

Even the machine beneath all of this is a loop. Fetch the next instruction, decode it, execute it, advance the counter: every processor ever shipped runs one loop, forever, and everything you have ever called "software" is just cargo riding inside it.

Here is why this tour matters and is not just a party trick. In every single example, you can point cleanly at the four layers. Cron's *flow* is the crontab; its *bodies* are your scripts; its *guarantee* is, notoriously, little more than an exit code and hope. The game engine's *guarantee* is the frame budget, enforced with a ruthlessness Apollo would recognize. The workflow engine's entire commercial value is that it took *flow* and *guarantee* (ordering, retries, timeouts, exactly-once-ish execution) out of your application code and made them someone else's rigorously tested problem. Different decades, different logos, same anatomy. The industry has been shipping the same four-layer machine since 1969 and arguing only about which layer to sell.

Which sets up the real question, the one the rest of this book exists to answer: if the anatomy never changes, what exactly is new now? Not the loop. The loop is fine. What is new is *who writes the inside of it*.

If you arrived with a different vocabulary

The tooling world has its own names for the things this series names, and it is worth mapping them once, because the four layers are underneath all of them. An **agent harness**, the fixed machinery that gives a model its tools, its state, its permissions, and its loop, is this book's subject from Chapter 9 onward, where it is called the exoskeleton: actor and flow held frozen around a generated body. **Evals** are the guarantee layer pointed at a component whose behavior is generated rather than written, and Chapter 12 is about what that costs when the thing under test is a distribution rather than a function. A **gauntlet loop**, run the agent until its output survives a battery of checks, is a stopping condition and a verifier bolted together; Chapter 10 is about the first half, Chapter 12 the second, and the question the pattern rarely asks, who wrote the checks and whether the thing being checked can see them, is the subject of Book III. And a **self-improving agent**, in the strong sense of a system that rewrites its own harness, is the one thing the series argues should not exist, on structural grounds that take until Book III, Chapter 13 to earn. The words change. The layers do not.

1.4 What "Frozen" Means

Time to make our central metaphor precise, because we are going to lean on it for five books.

Every piece of a software system belongs to one of three categories, and the sorting question is simply: *when does it come into existence?*

An **artifact** exists *before* execution. Someone wrote it, reviewed it, versioned it, deployed it. Your source code. The crontab. The DAG definition. The woven rope. Artifacts are **frozen**: at runtime they can be read and replayed but not created. Their defining virtue is that you can study them in advance (test them, prove things about them, print them out and defend them to a review board) because they hold still.

An **emission** exists *only during* execution. It is produced by the running system, in response to conditions no author fully foresaw: log lines, computed values, a game's particular frame, this particular HTTP response. Emissions are **fluid**. Classical software engineering's quiet genius was to arrange things so that emissions were *boring*: mere consequences, fully determined by frozen artifacts plus inputs. If an emission ever surprised you, that was called a bug.

A **guarantee** is the third thing, and it is neither. It is a *constraint or verifier applied to emissions*: the type check, the schema, the assertion, the priority table, the watchdog. Guarantees are **locked**: frozen artifacts, yes, but with a special role. They do not produce behavior; they *bound* it. When the rendezvous radar produced conditions no one had scripted, it was not the body that saved the landing. Bodies can only do what they were written to do. It was the locked layer.

Now the fulcrum of the whole series, stated plainly. In classical software, *body, actor, and flow are all artifacts.* All frozen. And this book, the one in your hands, is about applying a single transformation to the first of them:

```
G(x):  artifact(x)  ->  emission(x)
```

G for *generativize*: take something that used to be authored in advance and let the running system produce it. This book performs `G(body)`: the contents of each step, the decision about *what happens next*, becomes an emission, generated by a model, while actor and flow stay frozen and the guarantees stay locked. The sequels go further up the stack, and you can already guess how. But one melt at a time.

One more thing, before you object that your team already caches, templates, and hard-codes things back down after they have proven out: yes. Melting has an inverse. Proven emissions get refrozen into artifacts, and the mature systems in this series breathe in both directions. That operator gets its own chapter, two books from now, and a rocket to go with it. For now, the arrow points one way.

1.5 Where Guarantees Have Traditionally Come From

If classical software's behavior was frozen, where did its *trust* come from? It is worth taking inventory, because everything on this list was invented to guard authored behavior, and in later chapters we will ask, item by item, which of them survive contact with generated behavior.

Start at design time, where classical engineering does almost all of its guaranteeing. **Compilers and type systems** refuse, before a single instruction runs, entire categories of programs that could misbehave: a guarantee so absolute we stopped noticing it is one. **Tests** freeze expected behavior into executable claims: given this state, `f` must produce that state. **Schemas and contracts** pin down the shapes data may take at every boundary (a discipline whose absence once converted a Mars probe into a very expensive meteor; that story opens Chapter 3). **Code review** is a guarantee made of colleagues.

Then the runtime residue: thinner, humbler, and telling. **Assertions** die loudly rather than proceed wrongly. **Transactions** promise all-or-nothing, so a failure mid-flight cannot leave the world half-changed. **Watchdog timers**, the direct ancestors of Apollo's restart logic, sit outside the loop with one job: if the process stops making progress, kill it and recover, no questions asked. **Idempotency** makes retries safe; **checksums** make corruption visible; **monitoring** makes everything else visible to the people holding the pager.

Notice the proportions. Classical software front-loads its guarantees: the heavy machinery (types, tests, proofs, review) operates on the frozen artifacts *before* execution, and the runtime keeps only a light residue, because deterministic replay of verified artifacts does not need much guarding. The Apollo software was tested, simulated, and inspected into the ground for years; the restart logic was the thin last line, and in the event, the thin last line was enough, *because* everything behind it held still.

Now watch what this implies, because it is the engineering thesis of the entire series and we have arrived at it honestly: **the amount of guaranteeing you need at runtime is inversely proportional to how much of your behavior was frozen in advance.** Melt a layer, and the verification that used to happen at design time has nowhere to go but into the running system. Determinism does not disappear when the body becomes generated. It *relocates*: out of the behavior, into the guarantees. Chapter 12 will build that runtime machinery in earnest. Every chapter between here and there is preparation for it.

1.6 The First Crack in the Ice

So here is the classical machine, complete: a frozen body, executed by a frozen actor, choreographed by a frozen flow, bounded by locked guarantees. It landed on the Moon. It runs your bank. It is, by any fair accounting, the most successful engineering pattern of the last century.

And its limitation is the same fact as its virtue: *the body can only contain decisions somebody already made.* Every branch anticipated, every case enumerated, every "what next?" answered in advance by a person at a desk. For sixty years, when the world presented a situation the authors had not foreseen, software had exactly two moves: fail, or do the wrong thing confidently.

Chapter 2 makes the third move. We take the loop you are about to build, specifically one line of it, and replace the handwritten transition with an inference call:

```
state = f(state)        # a person wrote f, in advance
state = model(state)    # the step is generated, at runtime
```

One line. It looks like a refactor. It is a phase change: the first artifact in our stack becoming an emission, with everything that implies for errors, trust, and that inventory of guarantees you just took. But not yet. First you are going to build the frozen version, properly, with your own hands, because you cannot melt what you have not built, and because everything about it that seems boring today is a property you will spend the rest of this book fighting to keep.

1.7 Lab: A Loop You Can Trust

Meet **Meridian**, the company we will be keeping for most of the series: a mid-market subscription business, ten thousand customers, and a churn number that has lately given its executives a reason to take early lunches. Meridian will grow more elaborate as we go. Today it has ten accounts and one need: a churn report, produced by a loop so plain you could explain it to a review board on a napkin.

Everything in this lab is deliberately, pointedly deterministic. Same input, same output, every run, forever. Savor that. It is the last chapter where it comes free.

```
"""Meridian churn report -- a loop you can trust.

Book I, Chapter 1 lab. Deterministic. Boring. That's the point.
"""

from dataclasses import dataclass, replace
from enum import Enum

# ---------------------------------------------------------------
# STATE -- everything the loop knows, in one place, immutable.
# ---------------------------------------------------------------

class Phase(str, Enum):
    LOAD = "load"
    COMPUTE = "compute"
    SEGMENT = "segment"
    REPORT = "report"
    DONE = "done"

@dataclass(frozen=True)          # yes, frozen. The word will follow us around.
class State:
    phase: Phase = Phase.LOAD
```

```
    step: int = 0
    accounts: tuple = ()
    churn_rate: float | None = None
    segments: tuple = ()
    report: str | None = None

# ------------------------------------------------------------------
# THE BODY -- what happens during a step. Hand-written, every line.
# ------------------------------------------------------------------

RAW = (
    {"id": "A-01", "plan": "basic",      "cancelled": True},
    {"id": "A-02", "plan": "basic",      "cancelled": True},
    {"id": "A-03", "plan": "basic",      "cancelled": False},
    {"id": "A-04", "plan": "pro",        "cancelled": True},
    {"id": "A-05", "plan": "pro",        "cancelled": False},
    {"id": "A-06", "plan": "pro",        "cancelled": False},
    {"id": "A-07", "plan": "pro",        "cancelled": False},
    {"id": "A-08", "plan": "enterprise", "cancelled": False},
    {"id": "A-09", "plan": "enterprise", "cancelled": False},
    {"id": "A-10", "plan": "enterprise", "cancelled": False},
)

def load(s: State) -> State:
    return replace(s, phase=Phase.COMPUTE, accounts=RAW)

def compute(s: State) -> State:
    cancelled = sum(1 for a in s.accounts if a["cancelled"])
    return replace(s, phase=Phase.SEGMENT, churn_rate=cancelled /
  len(s.accounts))

def segment(s: State) -> State:
    plans = sorted({a["plan"] for a in s.accounts})
    segs = tuple(
        (p,
         sum(1 for a in s.accounts if a["plan"] == p and a["cancelled"]) /
         sum(1 for a in s.accounts if a["plan"] == p))
        for p in plans
    )
    return replace(s, phase=Phase.REPORT, segments=segs)

def report(s: State) -> State:
    lines = [f"MERIDIAN CHURN REPORT  |  overall: {s.churn_rate:.0%}"]
    lines += [f"  {plan:<12} {rate:>4.0%}" for plan, rate in s.segments]
    return replace(s, phase=Phase.DONE, report="\n".join(lines))

# ------------------------------------------------------------------
# THE FLOW -- how steps are arranged over time. Also handwritten.
# ------------------------------------------------------------------

FLOW = {
    Phase.LOAD: load,
    Phase.COMPUTE: compute,
```

```
    Phase.SEGMENT: segment,
    Phase.REPORT: report,
}

# ---------------------------------------------------------------
# THE GUARANTEES -- what must remain true, no matter what.
# ---------------------------------------------------------------

MAX_STEPS = 10   # the watchdog. Our rendezvous-radar insurance.

def run(state: State = State(), flow=FLOW):
    trail = []
    while True:
        if state.phase is Phase.DONE:                        # success condition
            trail.append(f"step {state.step}: TERMINATED -- success")
            break
        if state.step >= MAX_STEPS:                          # watchdog
            trail.append(f"step {state.step}: TERMINATED -- watchdog "
                         f"(step budget exhausted in phase
  '{state.phase.value}')")
            break
        body = flow[state.phase]                             # <- the flow,
  frozen
        nxt = body(state)                                    # <- the body,
  frozen
        assert isinstance(nxt.phase, Phase), "unknown phase"  # <- a guarantee
        trail.append(f"step {state.step}: {state.phase.value} ->
  {nxt.phase.value}")
        state = replace(nxt, step=state.step + 1)
    return state, trail
```

Before running it, walk the anatomy, because this file is the whole chapter in a hundred lines. The **state** is one frozen dataclass: every fact the loop knows, immutable, so each step's input and output can be captured whole.

The **body** is four pure functions, each taking a state and returning the next; note that they contain *all* of the intelligence and *none* of the control.

The **flow** is a table, literally a dictionary, mapping where-you-are to what-runs-next; the entire choreography of this program fits in four lines you can read aloud.

And the **guarantees** are three: a success condition, a step budget standing watch outside everything, and one assertion at the boundary. There is also a fourth, so pervasive it is easy to miss: the *trail*, an audit log built as the loop runs, because a loop you cannot replay is a loop you are merely hoping about.

Run 1, nominal:

```
--- RUN 1: nominal ---
step 0: load -> compute
step 1: compute -> segment
step 2: segment -> report
step 3: report -> done
step 4: TERMINATED -- success

MERIDIAN CHURN REPORT  |  overall: 30%
  basic         67%
  enterprise     0%
  pro           25%
```

Thirty percent churn, concentrated brutally in the basic tier. Meridian's executives were right to skip dessert. But the report is not the point of the lab. The point is Run 2, in which we re-create the rendezvous radar.

One component misbehaves, realistically and undramatically. The `compute` step does its arithmetic correctly but, through the kind of one-line oversight that survives code review every day, forgets to advance the phase:

```
def stuck_compute(s: State) -> State:
    cancelled = sum(1 for a in s.accounts if a["cancelled"])
    return replace(s, churn_rate=cancelled / len(s.accounts))   # phase unchanged. Oops.

SABOTAGED = {**FLOW, Phase.COMPUTE: stuck_compute}
```

Left alone, this loop violates the first obligation from section 1.1 (nothing changes each time around) and would run until the heat death of the process. It is not left alone:

```
--- RUN 2: one component misbehaves ---
step 0: load -> compute
step 1: compute -> compute
step 2: compute -> compute
step 3: compute -> compute
step 4: compute -> compute
step 5: compute -> compute
step 6: compute -> compute
step 7: compute -> compute
step 8: compute -> compute
step 9: compute -> compute
step 10: TERMINATED -- watchdog (step budget exhausted in phase 'compute')
report produced: None
```

Look at that trail. `compute -> compute -> compute`: a component drowning in place, exactly the signature the AGC's executive saw in 1969. And then the locked layer doing its one job: not fixing the fault, not understanding it, just *refusing to let it*

run forever, and leaving behind an honest account of what happened and where. The body failed. The guarantee held. Your first loop just survived its own 1202, and the flight recorder can prove it.

Two closing instructions for the lab, one practical, one in pencil.

Practical: extend it. Add a `budget_ms` guarantee alongside `MAX_STEPS`. Add a second termination reason. Serialize the final `State` to JSON and confirm a fresh process can resume from it. You have just implemented replay, and you will want it forever.

And in pencil: circle these two lines.

```
        body = flow[state.phase]     # <— the flow, frozen
        nxt = body(state)            # <— the body, frozen
```

The second line is where Chapter 2 applies the torch. The first is where Book III does. Keep the file.

Where We've Landed

A loop is a rule applied to state until a condition holds: the one structure that turns finite text into open-ended work, carrying two obligations, progress each step and a reachable end. Every loop, from cron to Eagle, decomposes into the same four layers: **body** (what a step does), **actor** (who does it), **flow** (how steps are arranged), **guarantee** (what must remain true).

In classical software all of the first three are **artifacts**, frozen before execution, which is precisely why its runtime guarantees could stay thin, why its emissions stayed boring, and why a young man in Houston could say GO: the entire trust model of twentieth-century software rests on behavior that holds still.

And the series' single transformation is now on the table: `G(x)` turns an artifact into an emission, and this book performs it on the body, knowing that whatever determinism we melt out of the behavior must be rebuilt, at runtime, in the guarantees.

What's Next

In 1966, while Böhm and Jacopini were proving what loops could do, a computer scientist at MIT wrote a few hundred lines of pattern-matching code and discovered, to his lasting horror, that people would pour their hearts out to it. In the next chapter we put a model inside the loop, and find out why the hardest part is not making it work, but knowing what "working" now means.

Steve Bales said GO because he could finish the sentence "the machine will now…" with every clause of it written by someone he could name. The rest of this book is about earning that sentence back after we hand the pen to the machine.